

Course: DD2424 - Assignment 1

In this assignment you will train and test a one layer network with multiple outputs to classify images from the CIFAR-10 dataset. You will train the network using mini-batch gradient descent applied to a cost function that computes the cross-entropy loss of the classifier applied to the labelled training data and an L_2 regularization term on the weight matrix.

Background 1: Mathematical background

The mathematical details of the network are as follows. Given an input vector, $\mathbf{x}$, of size $d \times 1$ our classifier outputs a vector of probabilities, $\mathbf{p}$ ($K \times 1$), for each possible output label:

$$\mathbf{s} = W\mathbf{x} + \mathbf{b} \quad (1)$$

$$\mathbf{p} = \text{SOFTMAX}(\mathbf{s}) \quad (2)$$

where the matrix W has size $K \times d$, the vector $\mathbf{b}$ is $K \times 1$ and SOFTMAX is defined as

$$\text{SOFTMAX}(\mathbf{s}) = \frac{\exp(\mathbf{s})}{\mathbf{1}^T \exp(\mathbf{s})} \quad (3)$$

The predicted class corresponds to the label with the highest probability:

$$k^* = \arg \max_{1 \leq k \leq K} \{p_1, \dots, p_K\} \quad (4)$$

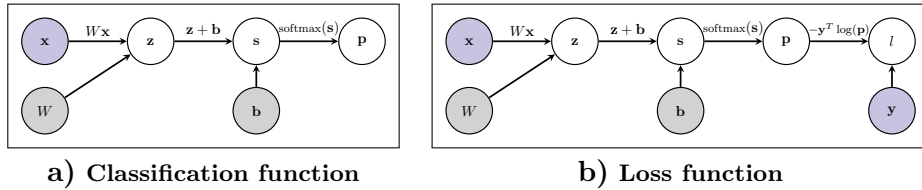


Figure 1: Computational graph of the classification and loss function that is applied to each input $\mathbf{x}$ in this assignment.

The classifier's parameters W and $\mathbf{b}$ are what we have to learn from the labelled training data. Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, with each $y_i \in \{1, \dots, K\}$ and $\mathbf{x}_i \in \mathbb{R}^d$, represent our labelled training data. In the lectures we have described how to set the parameters by minimizing the cross-entropy loss plus a regularization term on W . Mathematically this cost function is

$$J(\mathcal{D}, \lambda, W, \mathbf{b}) = \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{x}, y) \in \mathcal{D}} l_{\text{cross}}(\mathbf{x}, y, W, \mathbf{b}) + \lambda \sum_{i,j} W_{ij}^2 \quad (5)$$

where

$$l_{\text{cross}}(\mathbf{x}, y, W, \mathbf{b}) = -\log(p_y) \quad (6)$$

and $\mathbf{p}$ has been calculated using equations (1, 2). (Note if the label is encoded by a one-hot representation then the cross-entropy loss is defined as $-\mathbf{y}^T \log(\mathbf{p}) = \log(\mathbf{p}_y)$.) The optimization problem we have to solve is

$$W^*, \mathbf{b}^* = \arg \min_{W, \mathbf{b}} J(\mathcal{D}, \lambda, W, \mathbf{b}) \quad (7)$$

In this assignment (as described in the lectures) we will solve this optimization problem via mini-batch gradient descent.

For mini-batch gradient descent we begin with a sensible random initialization of the parameters $W, \mathbf{b}$ and we then update our estimate for the parameters with

$$W^{(t+1)} = W^{(t)} - \eta \left. \frac{\partial J(\mathcal{B}^{(t+1)}, \lambda, W, \mathbf{b})}{\partial W} \right|_{W=W^{(t)}, \mathbf{b}=\mathbf{b}^{(t)}} \quad (8)$$

$$\mathbf{b}^{(t+1)} = \mathbf{b}^{(t)} - \eta \left. \frac{\partial J(\mathcal{B}^{(t+1)}, \lambda, W, \mathbf{b})}{\partial \mathbf{b}} \right|_{W=W^{(t)}, \mathbf{b}=\mathbf{b}^{(t)}} \quad (9)$$

where η is the learning rate and $\mathcal{B}^{(t+1)}$ is called a mini-batch and is a random subset of the training data $\mathcal{D}$ and

$$\frac{\partial J(\mathcal{B}^{(t+1)}, \lambda, W, \mathbf{b})}{\partial W} = \frac{1}{|\mathcal{B}^{(t+1)}|} \sum_{(\mathbf{x}, y) \in \mathcal{B}^{(t+1)}} \frac{\partial l_{\text{cross}}(\mathbf{x}, y, W, \mathbf{b})}{\partial W} + 2\lambda W \quad (10)$$

$$\frac{\partial J(\mathcal{B}^{(t+1)}, \lambda, W, \mathbf{b})}{\partial \mathbf{b}} = \frac{1}{|\mathcal{B}^{(t+1)}|} \sum_{(\mathbf{x}, y) \in \mathcal{B}^{(t+1)}} \frac{\partial l_{\text{cross}}(\mathbf{x}, y, W, \mathbf{b})}{\partial \mathbf{b}} \quad (11)$$

To compute the relevant gradients for the mini-batch, we then have to compute the gradient of the loss w.r.t. each training example in the mini-batch. You should refer to the lecture notes for the explicit description of how to compute these gradients.

Before Starting

I assume that you will complete the assignment in *Matlab*. You can complete the assignment in another programming language. If you do though I will not answer programming specific questions and you will also probably have to find a way to display, plot and graph your results.

Besides invoking *Matlab* commands, you will be required to run a few operating system commands. For these commands I will assume your computer's

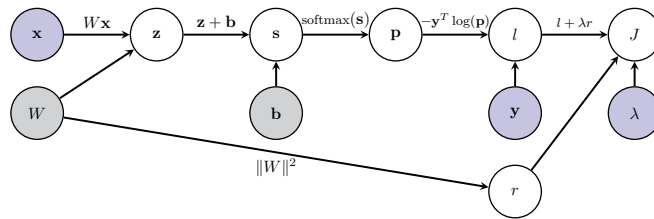


Figure 2: Computational graph of the cost function applied to a mini-batch containing one training example $\mathbf{x}$. If you have a mini-batch of size greater than one, then the loss computations are repeated for each entry in the mini-batch (as in the above graph), but the regularization term is only computed once.

operating system is either linux or unix. If otherwise, you'll have to fend for yourself. But all the non-*Matlab* commands needed are more-or-less trivial.

The notes for this assignment, and those to follow, will give you pointers about which *Matlab* commands to use. However, I will not give detailed explanations about their usage. I assume you have some previous experience with *Matlab*, are aware of many of the in-built functions, how to manipulate vectors and matrices and how to write your own functions etc. Keep in mind the function `help` can be called to obtain information about particular functions. So for example

```
>> help plot
```

will display information about the *Matlab* command `plot`.

Background 2: *Getting Started*

Set up your environment

Create a new directory to hold all the *Matlab* files you will write for this course:

```
$ mkdir DirName
$ cd DirName
$ mkdir Datasets
$ mkdir Result_Pics
```

Download the CIFAR-10 dataset stored in its *Matlab* format from [this link](#). Move the `cifar-10-matlab.tar.gz` file to the `Datasets` directory you have just created, `untar` the file and then move up to the parent directory. Also download the file `montage.m` from the Canvas webpage for Assignment 1 and move it to `DirName`.

```
$ mv montage.m DirName/
$ mv cifar-10-matlab.tar.gz DirName/Datasets
$ cd DirName/Datasets
$ tar xvfz cifar-10-matlab.tar.gz
$ cd ..
```

Background 3: *Useful Display Function*

You have copied a function called `montage.m`, which is a slightly modified version of the function available at:

<http://www.mathworks.com/matlabcentral/fileexchange/22387>

This is a useful function as it allows you to efficiently view the images in a directory or in a *Matlab* array or a cell array. To look at some of the images from the CIFAR-10 dataset you can use the following set of commands:

```
>> addpath DirName/Datasets/cifar-10-batches-mat/;
>> A = load('data_batch_1.mat');
>> I = reshape(A.data', 32, 32, 3, 10000);
>> I = permute(I, [2, 1, 3, 4]);
>> montage(I(:, :, :, 1:500), 'Size', [5,5]);
```

This sequence of commands tells *Matlab* to add the directory of the CIFAR-10 dataset to its path. Then it loads one of the `.mat` files containing image and label data. You access the image data with the command `A.data`. It has size 10000×3072 . Each row of `A.data` corresponds to an image of size $32 \times 32 \times 3$ that has been flattened into a row vector. We can re-arrange `A.data` into an array format expected by `montage` ($32 \times 32 \times 3 \times 10000$) using the command `reshape`. After reshaping the array, the rows and columns of each image still need to be permuted and this is achieved with the `permute` command. Now you have an array format `montage` expects. In the above code we just view the first 500 images. Use `help` to find out the different ways `montage` can be called.

You have looked at some of the CIFAR-10 images. Now it is time to start writing some code.

Exercise 1: *Training a multi-linear classifier*

For this assignment you will just use data in the file `data_batch_1.mat` for training, the file `data_batch_2.mat` for validation and the file `test_batch.mat` for testing. Create a file `Assignment1.m`. In this file you will write the code

for this assignment and the necessary (sub-)functions. Here are my recommendations for which functions to write and the order in which to write them:

1. Write a function that reads in the data from a CIFAR-10 batch file and returns the image and label data in separate files. Make sure to convert your image data to `single` or `double` format. I would suggest the function has the following input and outputs

```
function [X, Y, y] = LoadBatch(filename)
```

where

- `X` contains the image pixel data, has size $d \times n$, is of type `double` or `single` and has entries between 0 and 1. `n` is the number of images (10000) and `d` the dimensionality of each image ($3072=32 \times 32 \times 3$).
- `Y` is $K \times n$ ($K = \#$ of labels = 10) and contains the one-hot representation of the label for each image.
- `y` is a vector of length n containing the label for each image. A note of caution. CIFAR-10 encodes the labels as integers between 0-9 but *Matlab* indexes matrices and vectors starting at 1. Therefore it may be easier to encode the labels between 1-10.

This function will not be long. You just need to call `A = load(fname);` and then rearrange and store the values in `A.data` and `A.labels`.

Top-level: Read in and store the training, validation and test data.

2. Next we should pre-process the raw input data as it helps training. You should transform training data to have zero mean. If `trainX` is the $d \times n$ image data matrix (each column corresponds to an image) for the training data then

```
mean_X = mean(trainX, 2);
std_X = std(trainX, 0, 2);
```

Both `mean_X` and `std_X` have size $d \times 1$.

You should normalize the training, validation and test data with respect to the mean and standard deviation values computed from the training data as follows. If `X` is an $d \times n$ image data matrix then you can normalize `X` as

```
X = X - repmat(mean_X, [1, size(X, 2)]);
X = X ./ repmat(std_X, [1, size(X, 2)]);
```

Top-level: Compute the mean and standard deviation vector for the training data and then normalize the training, validation and test data w.r.t. these mean and standard deviation vectors.

3. **Top-Level:** After reading in and pre-processing the data, you can initialize the parameters of the model W and b as you now know what size they should be. W has size $K \times d$ and b is $K \times 1$. Initialize each entry to have Gaussian random values with zero mean and standard deviation .01. You should use the Matlab function `randn` to create this data.
4. Write a function that evaluates the network function, i.e. equations (1, 2), on multiple images and returns the results. I would suggest the function has the following form

```
function P = EvaluateClassifier(X, W, b)
```

where

- each column of X corresponds to an image and it has size $d \times n$.
- W and b are the parameters of the network.
- each column of P contains the probability for each label for the image in the corresponding column of X . P has size $K \times n$.

Top-level: Check the function runs on a subset of the training data given a random initialization of the network's parameters:

```
P = EvaluateClassifier(trainX(:, 1:100), W, b).
```

5. Write the function that computes the cost function given by equation (5) for a set of images. I suggest the function has the following inputs and outputs

```
function J = ComputeCost(X, Y, W, b, lambda)
```

where

- each column of X corresponds to an image and X has size $d \times n$.
- each column of Y ($K \times n$) is the one-hot ground truth label for the corresponding column of X or Y is the $(1 \times n)$ vector of ground truth labels.
- J is a scalar corresponding to the sum of the loss of the network's predictions for the images in X relative to the ground truth labels and the regularization term on W .

6. Write a function that computes the accuracy of the network's predictions given by equation (4) on a set of data. Remember the accuracy of a classifier for a given set of examples is the percentage of examples for which it gets the correct answer. I suggest the function has the following inputs and outputs

```
function acc = ComputeAccuracy(X, y, W, b)
```

where

- each column of X corresponds to an image and X has size $d \times n$.
- y is the vector of ground truth labels of length n .
- acc is a scalar value containing the accuracy.

7. Write the function that evaluates, for a mini-batch, the gradients of the cost function w.r.t. W and b , that is equations (10, 11). I suggest the function has the form

```
function [grad_W, grad_b] = ComputeGradients(X, Y, P, W, lambda)
```

where

- each column of X corresponds to an image and it has size $d \times n$.
- each column of Y ($K \times n$) is the one-hot ground truth label for the corresponding column of X .
- each column of P contains the probability for each label for the image in the corresponding column of X . P has size $K \times n$.
- $grad_W$ is the gradient matrix of the cost J relative to W and has size $K \times d$.
- $grad_b$ is the gradient vector of the cost J relative to b and has size $K \times 1$.

Be sure to check out how you can efficiently compute the gradient for a batch from the last slide of Lecture 3. This can lead to a much faster implementation (> 3 times faster) than looping through each training example in the batch.

Everyone makes mistakes when computing gradients. Therefore you must always check your analytic gradient computations against numerical estimations of the gradients! Download code from the Canvas webpage that computes the gradient vectors numerically. Note there are two versions 1) a slower but more accurate version based on the *centered difference* formula and 2) a faster but less accurate based on the *finite difference method*. You will probably have to make small changes to the functions to make them compatible with your code. It will take some time to run the numerical gradient calculations as your network has $32 \times 32 \times 3 \times 10$ different parameters in W . Initially, you should just perform your checks on mini-batches of size 1 and with no regularization ($\lambda=0$). Afterwards you can increase the size of the mini-batch and include regularization into the computations. Another trick is that you should send in a reduced dimensionality version of $trainX$ and W , so instead of

```
[ngrad_b, ngrad_W] = ComputeGradsNumSlow(trainX(:, 1), trainY(:, 1),
W, b, lambda, 1e-6);
```

you can compute the gradients numerically for smaller inputs (the first 20 dimensions of the first training example)

```
[ngrad_b, ngrad_W] = ComputeGradsNumSlow(trainX(1:20, 1), trainY(:, 1),
W(:, 1:20), b, lambda, 1e-6);
```

You should then also compute your analytical gradients on this reduced version of the input data with reduced dimensionality. This will speed up computations and also reduce the risk of numerical precision issues (very possible when the full W is initialized with very small numbers and `trainX` also contains small numbers).

You could compare the numerically and analytically computed gradient vectors (matrices) by examining their absolute differences and declaring, if all these absolute difference are small ($<1e-6$), then they have produced the same result. However, when the gradient vectors have small values this approach may fail. A more reliable method is to compute the relative error between a numerically computed gradient value g_n and an analytically computed gradient value g_a

$$\frac{|g_a - g_n|}{\max(\text{eps}, |g_a| + |g_n|)} \quad \text{where eps a very small positive number}$$

and check this is small. There are potentially more issues that can plague numerical gradient checking (especially when you start to train deep rectifier networks), so I suggest you read the relevant section of the [Additional material for lecture 3](#) from Stanford's course **Convolutional Neural Networks for Visual Recognition** for a more thorough exposition especially for the subsequent assignments.

Do not continue with the rest of this assignment until you are sure your analytic gradient code is correct. If you are having problems, set the seed of the random number generator with the command `rng` to ensure at each test W and b have the same values and double/triple check that you have a correct implementation of the gradient equations from the lecture notes.

8. Once you have the gradient computations debugged you are now ready to write the code to perform the mini-batch gradient descent algorithm to learn the network's parameters where the updates are defined in equations (8, 9). You have a couple of parameters controlling the learning algorithm (for this assignment you will just implement the most vanilla version of the mini-batch gradient descent algorithm, with no adaptive tuning of the learning rate or momentum terms):

- `n_batch` the size of the mini-batches
- `eta` the learning rate
- `n_epochs` the number of runs through the whole training set.

As the images in the CIFAR-10 dataset are in random order, the easiest to generate each mini-batch is to just run through the images sequentially. Let `n_batch` be the number of images in a mini-batch. Then for one epoch (a complete run through all the training images), you can generate the set of mini-batches with this snippet of code:

```

for j=1:n/n_batch
    j_start = (j-1)*n_batch + 1;
    j_end = j*n_batch;
    inds = j_start:j_end;
    Xbatch = Xtrain(:, j_start:j_end);
    Ybatch = Ytrain(:, j_start:j_end);
end

```

(A slight upgrade of this default implementation is to randomly shuffle your training examples before each epoch. One efficient way to do this is via the command `randperm` which when given the input `n` returns a vector containing a random permutation of the integers `1:n`.) I suggest the mini-batch learning function has these inputs and outputs

```
function [Wstar, bstar] = MiniBatchGD(X, Y, GDparams, W, b, lambda)
```

where `X` contains all the training images, `Y` the labels for the training images, `W`, `b` are the initial values for the network's parameters, `lambda` is the regularization factor in the cost function and

- `GDparams` is an object containing the parameter values `n_batch`, `eta` and `n_epochs`

For my initial experiments I set `n_batch=100`, `eta=.001`, `n_epochs=20` and `lambda=0`. To help you debug I suggest that after each epoch you compute the cost function and print it out (and save it) on all the training data. For these parameter settings you should see that the training cost decreases for each epoch. After the first epoch my cost score on all the training data was 1.981428 where I had set the random number seed generator to `rng(400)` and I had initialized the weight matrix before the bias vector. In figure 8 you can see the training cost score when I run these parameter settings for 40 epochs. The cost score on the validation set is plotted in red in the same figure.

(Note: in `Tensorflow` and other software packages they count in the number of update steps as opposed to the number of training epochs. Given `n` training images and mini-batches of size `n_batch` then one epoch will result in `n/n_batch` update steps of the parameter values. Obviously, the performance of the resulting network depends on the number of update steps and how much training data is used. Therefore to make fair comparisons one should make sure the number of update steps is consistent across runs - for instance if you change the size of the mini-batch, number of training images, etc.. you may need to run more or fewer epochs of training.)

When you have finished training you can compute the accuracy of your learnt classifier on the test data. My network achieves (after 40



Figure 3: The graph of the training and validation loss computed after every epoch. The network was trained with the following parameter settings: `n_batch=100`, `eta=.001`, `n_epochs=40` and `lambda=0`.

epochs) an accuracy of 38.83% (with a random shuffling of the training example at the beginning of each epoch). Much better than random but not great. Hopefully, we will achieve improved accuracies in the assignments to come when we build and train more complex networks.

After training you can also visualization the weight matrix W as an image and see what *class templates* your network has learnt. Figure 4 shows the templates my network learnt. Here is a code snippet to re-arrange each row of W (assuming W is a $10 \times d$ matrix) into a set of images that can be displayed by *Matlab*.

```
for i=1:10
    im = reshape(W(i, :), 32, 32, 3);
    s_im{i} = (im - min(im(:))) / (max(im(:)) - min(im(:)));
    s_im{i} = permute(s_im{i}, [2, 1, 3]);
end
```

Then you can use either `imshow`, `imagesc` or `montage` to display the images in the cell array `s_im`.

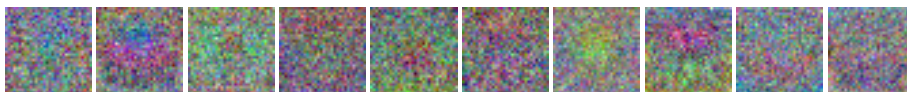


Figure 4: The learnt W matrix visualized as class template images. The network was trained with the following parameter settings: `n_batch=100`, `eta=.001`, `n_epochs=40` and `lambda=0`.

To complete the assignment:

In the DD2424 Canvas Assignment 1 page upload:

- The final (and cleaned up) version of the code you have written for the assignment. If you have written the code in multiple files please place

them in one file for the uploaded version. This will make life easier for me and the TAs! Note we will not run your code.

- A pdf document containing a short report. Please **do not zip** your code and pdf document into one file and upload the zip file. Then the graders have to download the zip file and uncompress it and this is time consuming. Here you should state whether you successfully managed to write the functions to correctly compute the gradient analytically, what tests you ran to check against the numerically computed gradient and the results of these tests. You should include the following plots/figures

1. Graphs of the loss and the cost function (when $\lambda = 0$, these two quantities are the same) on the training data and the validation data after each epoch of the mini-batch gradient descent algorithm.
2. Images representing the learnt weight matrix after the completion of training.

for the following parameter settings

- $\lambda=0$, $n_epochs=40$, $n_batch=100$, $\eta=.1$
- $\lambda=0$, $n_epochs=40$, $n_batch=100$, $\eta=.001$
- $\lambda=.1$, $n_epochs=40$, $n_batch=100$, $\eta=.001$
- $\lambda=1$, $n_epochs=40$, $n_batch=100$, $\eta=.001$

You should also report the final test accuracy your network achieves after each of these training runs. You should also make a short comment on the effect of increasing the amount of regularization and the importance of the correct learning rate.

Exercise 2: *Optional for Bonus Points*

1. **Improve performance of the network** It would be interesting to discover if it is possible to improve on the performance achieved in the first part of the assignment. Here are some tricks/avenues you can explore to help bump up performance (most of the tricks you can use for higher capacity networks and they will probably have much more of an effect for the higher capacity networks where over-fitting is more of a problem).:
 - (a) Use all the available training data for training (all five batches minus a small subset of the training images for a validation set). Decrease the size of the validation set down to ~ 1000 .
 - (b) Augment your training data by flipping an image horizontally with a .5 probability each time you encounter it during training. You can implement flipping efficiently by pre-computing the indices of the entries of the image vector that you have to switch. In **Matlab** given how a **Cifar-10** image is stored as a column vector, **im**, of size 3072×1 then one can flip the image horizontally with

```

    im_flipped = im(ind.flips);
where
    aa = 0:1:31;
    bb = 32:-1:1;
    vv = repmat(32*aa, 32, 1);
    ind_flip = vv(:) + repmat(bb', 32, 1);
    inds_flip = [ind_flip; 1024+ind_flip; 2048+ind_flip];

```

For those of you writing your code in `python`, the array `inds_flip` might be different as the image pixel data in `im` might be stored in a different order to `Matlab`. You should visualize your data-augmentation applied to individual images to make sure your code is doing the correct thing. Training with this augmentation should at the very least reduce the amount of over-fitting (gap between training and test performance). Also you should probably reduce the amount of L_2 regularization you apply by reducing the value of λ when you combine it with data-augmentation.

- (c) Do a grid search to find *good* values for the amount of L_2 regularization, the learning rate and the batch size. There is some empirical evidence that training with smaller batch sizes when using SGD training leads to better generalization. Though there is also more empirical evidence that with larger batch sizes you can increase the learning rate and not suffer as much with over-fitting... However, due to the simplicity of the model not too much over-fitting will be observed in any case.
- (d) Play around with decaying the learning rate by a factor of 10 after every n th epoch or when the validation seems to plateau. This is known as step decay.

Bonus Points Available: 1 point (if you complete at least 3 improvements - you can follow my suggestions and/or think of your own or some combination.)

2. Train network - multiple binary cross-entropy losses

In the assignment you just completed the softmax operation was used to turn the output scores $\mathbf{s}$ into a probability vector $\mathbf{p}$. This operation ensures the entries of $\mathbf{p}$ sum to one and explicitly assumes that only one class is present in the image. However, this is not the only option to turn $\mathbf{s}$ into a vector of probabilities $\mathbf{p}$. One can instead interpret that each entry p_i in $\mathbf{p}$ should correspond to the probability that class i is present in the image and is independent of the probabilities for the other classes. In this interpretation each $0 \leq p_i \leq 1$ for $i = 1, \dots, K$, but there is no constraint that the p_i 's sum to one. When this weaker assumption is made, the sigmoid function, $\sigma : \mathbb{R} \rightarrow [0, 1]$:

$$\sigma(s) = \frac{1}{1 + \exp(-s)} = \frac{\exp(s)}{\exp(s) + 1} \quad (12)$$

is usually used to map a score to a number between 0 and 1. The sigmoid is applied element wise to $\mathbf{s}$

$$\mathbf{p} = \sigma(\mathbf{s}) \quad (13)$$

Given this interpretation of the probability vector, it is usual to use K binary cross-entropy losses for training the network. For an input $\mathbf{x}$ with one-hot encoding, $\mathbf{y}$, of its label this multiple binary cross-entropy loss is defined as:

$$l_{\text{multiple bce}}(\mathbf{x}, \mathbf{y}) = -\frac{1}{K} \sum_{k=1}^K [(1 - y_k) \log(1 - p_k) + y_k \log(p_k)] \quad (14)$$

For this bonus point assignment you should train your network, with the softmax operation replaced by the sigmoid function, using the multiple binary cross-entropy loss. Check what the new gradient is and if you have to change your code or not. In the report:

- Please write down the expression for $\partial l_{\text{multiple bce}} / \partial \mathbf{s}$
- Record the final test accuracy you achieve training with this loss where a classification prediction is still based on the highest output probability as defined in equation (4). Note for training you may have to increase the learning rate because of the $1/K$ factor in the loss.
- For the test data make a histogram plot of the probability for the ground truth class for the examples correctly and those incorrectly classified. Check if there is a qualitative difference between these histograms when training is performed with the new training procedure versus the softmax + cross-entropy training. Also check if there is more or less over-fitting by looking at the training and validation loss plots.

Bonus Points Available: 2 points

To get the bonus point(s) you must upload the following to the Canvas page *Assignment 1 Bonus Points*:

1. Your code.
2. A Pdf document which
 - reports on your trained network with the best test accuracy, what improvements you made and which ones brought the largest gains. (Exercise 2.1)
 - compares the test accuracy of the network trained with the multiple binary cross-entropy loss compared to the cross-entropy loss and if more or less over-fitting occurs with this loss compare to the cross-entropy loss (Exercise 2.2).